

Homework 0

*Instructor: Eysa Lee**Due: September 30, 2026*

Throughout the semester you will be implementing (basic versions of) cryptographic systems we discuss in class. This assignment is meant to familiarize you with the structure of the programming assignments and the submission platform. If you have any questions, please do not hesitate to ask the teaching staff for support.

1 AI and External Resources Policy

You are permitted the use of AI to assist in syntax and debugging your program assignments. However, you are *not* allowed to have AI code up the entire assignment for you. You are also not copy code from libraries, implementations, or elsewhere. You should, using your knowledge from class and course materials, come up with the design of your program without the assistance of AI or other external resources. Violation of this policy is considered an honor code violation and may result in a failing grade for the assignment (and possibly the entire course).

2 Assignment Specification

In this assignment you will be implementing four basic cryptographic schemes we have discussed in class: Diffie-Hellman key exchange, El-Gamal encryption, and RSA encryption. In your code, you will define and implement the following 5 functions:

- `diffie_hellman`
- `elgamal_encrypt`
- `elgamal_decrypt`
- `rsa_encrypt`
- `rsa_decrypt`

For testing and grading, you will also submit a “main” program that can be used to interface with your implementation. Your “main” program may be in any language, but the your implementation for this assignment must be in Rust or C++. (It will likely be easiest to have your main program written in the same language as your implementation.)

2.1 Main Program API Specification

The main program will read a sequence of newline delineated commands from standard input ending with EOF. When instructed to, it will display output to standard out. We will only use the main program in the way specified in the API specification and with valid inputs. We are not looking to trip you up on API edge cases. When you turn in your assignment, please also turn in a one line file that contains the command to run the main program. For simplicity, input messages will be given as integers.

Command	Behavior
setup_params [p] [g] [q]	This command is for setting up the public parameters needed for the Diffie-Hellman Key Exchange and El-Gamal. We will run this first to specify the modulus p , group generator g , and group order q . It does not need to produce an output.
rsa_encrypt [N] [publicKey] [message]	Given an RSA modulus N and a public key, output an RSA encryption of a message m .
rsa_decrypt [N] [secretKey] [ciphertext]	Given an RSA modulus N and a secret key, output the decryption of a ciphertext.
dh_key_exchange [publicKeyA]	This command should perform a Diffie-Hellman Key Exchange with input publicKeyA as g^{k_a} . It should locally sample Party B's secret key k_b , compute Party B's public key as g^{k_b} , and compute the resulting shared key as $g^{k_a \cdot k_b}$. This command should output (on separate lines) the values k_b and $g^{k_a \cdot k_b}$.
elgamal_encrypt [publicKey] [message]	Given a public key pk and a message m , this should output (on separate lines) the values c_1 and c_2 , where (c_1, c_2) is an El-Gamal encryption of m under pk .
elgamal_decrypt [secretKey] [c1] [c2]	Given a secret key sk and a ciphertext tuple (c_1, c_2) , where c_1, c_2 , this should output onto standard out the decryption m of the ciphertext.

2.2 External Libraries

Allowed. This assignment requires modular arithmetic over integers, and you are allowed to use existing libraries for easier representations of large integers.

Not Allowed. There are many external libraries that already implement the functions in this assignment. It should go without saying, but you are *not* allowed to use existing implementations of Diffie-Hellman Key Exchange, El-Gamal, or RSA. That is, your code must do the steps of the algorithms that we have seen in class rather than just calling another library that already implements it.

If you are using an external library for large integers, they may have a specialized function for computing the modular inverse and exponentiation. For computing the modular inverse, you are expected to implement this yourself. For more efficient exponentiation (i.e., if you are not using the built-in operation for exponentiation), you are also expected to implement this yourself.

2.3 Submission

Your assignment should be submitted through Gradescope as a ZIP. You will submit the following four components:

1. Source code for your implementation (written in either Rust or C++).
2. A Makefile that compiles your implementation upon running `make`.
3. A main program and file titled “`COMMAND`” that contains on one line the command to run your main program.
4. A PDF titled “design” that includes your name, UNI, and instructions for how to run your implementation by hand. In future assignments, this document will also include details of your implementation, major design decisions, and any other information you think is relevant to the grading of your assignment.

Codewalks. There will be random codewalks throughout the semester where you will be expected to explain not only your code but also design decisions. Codewalks are only announced after submissions close.